

# 期末设计-数字时钟

计算机学院 2313725 张耕嘉

## 一、设计思路

**时钟信号 (clock):** 用于触发计数器的计数操作。设时钟信号频率为 50MHz, 通过分频器可以得到每秒钟触发一次的时钟信号 (2.98 Hz)。

**复位信号 (reset\_n):** 用于将计数器清零。当复位信号为高电平时, 秒计数器、分钟计数器和小时计数器都会被清零。

**使能信号 (enable):** 用于控制计数器是否开始计数。当使能信号为高电平时, 计数器开始计数; 当使能信号为低电平时, 计数器停止计数。

**加载数据模式信号 (load):** 用于进入加载数据模式。当加载数据模式信号为高电平时, 我们可以按下设置按钮来加载新的时间数据。

**设置信号 (setting1/2/3):** 分别用于加载秒数据、分钟数据和小时数据。当设置信号为高电平时, 在加载数据模式下, 按下对应的设置按钮可以加载相应的时间数据。

对于秒计数器, 它接收秒数据信号作为输入, 并输出当前的秒数。在每次时钟上升沿触发时, 根据各种条件进行判断和操作, 包括复位信号、加载数据模式信号和设置信号等。具体的逻辑如下:

- 1) 当复位信号为高电平时, 秒计数器和进位控制信号都被清零。
- 2) 当加载数据模式信号和设置信号均为高电平时, 如果秒计数器小于 59, 则秒计数器加 1; 如果秒计数器等于 59, 则秒计数器被清零。
- 3) 当秒计数器等于 59 且加载数据模式信号为低电平时, 秒计数器被清零, 并启用分钟计数器的计数。
- 4) 当秒计数器等于 58 且加载数据模式信号为低电平时, 秒计数器加 1, 并禁用分钟计数器的计数。
- 5) 当秒计数器小于 58 且使能信号为高电平且加载数据模式信号为低电平时, 秒计数器加 1。

对于分钟计数器和小时计数器, 它们的实现原理与秒计数器类似。分钟计数器接收秒计数器的进位信号作为使能信号, 并输出当前的分钟数。小时计数器接收分钟计数器的进位信号作为使能信号, 并输出当前的小时数。

- 1) 当分钟计数器的进位信号为高电平且加载数据模式信号为低电平时, 分钟计数器被清零, 并启用小时计数器的计数。
- 2) 当分钟计数器为 59 且使能信号为高电平且加载数据模式信号为低电平时, 分钟计数器被清零, 并启用小时计数器的计数。
- 3) 当分钟计数器小于 59 且使能信号为高电平且加载数据模式信号为低电平时, 分钟计数器加 1。

最后, 顶层模块将这三个计数器模块连接在一起, 并通过中间信号进行通信。秒计数器的进位信号被连接到分钟计数器的使能信号, 分钟计数器的进位信号被连接到小时计数器的使能信号, 以实现级联计数。

## 二、功能代码

## 1、秒计数器模块

```
module counter_sec(
    input clock,      // 时钟信号(50MHz) 通过分频器得到每秒钟触发一次的时钟信号
                      //  $(50\text{MHz}) / (2^{24}) = 2.98\text{ Hz}$ 
    input enable_sec, // 使能信号, 表示计数器是否开始计数
    input reset_sec,  // 复位信号, 在高电平状态下将计数器清零。
    input load_sec,   // 加载数据模式, 当接收到高电平信号时, 启用加载数据模式,
                      // 此时按下设置按钮可以加载新的秒数据。
    input setting_sec, // 设置信号, 当接收到高电平信号时, 表示需要进行时间设置
                      // 操作。
    input [5:0] data_sec, // 秒数据信号, 6 位二进制数, 用于设置计数器初始值。
    output reg [5:0] count_sec, // 计数器输出信号, 6 位二进制数, 表示当前的秒数。
    output reg carry_sec, // 进位信号, 当秒计数器达到 59 时, 进位信号会被置
                      // 为高电平, 并且允许分钟计数器执行加 1 操作。
    output reg carry_sec1 // 进位信号, 当计时器达到 00:59:59 时, 进位信号会被置
                      // 为高电平, 并且允许小时计数器执行加 1 操作 00:59:59 → 01:00:00。
);

    always @ (posedge clock or posedge reset_sec )
    begin
        if (reset_sec)begin
            count_sec<=6'b000000; // 复位秒计数器和进位控制信号
            carry_sec<=1'b0;
        end

        else if (load_sec && setting_sec && count_sec<6'd59 ) begin
            count_sec<=count_sec+1; // 按钮按下: load+1 = sec+1
        end

        else if (load_sec && setting_sec && count_sec==6'd59 ) begin
            count_sec<=6'b000000; // 按钮按下: load+1 = 59→00
        end

        else if (count_sec==6'd59 && load_sec==0 ) begin
            count_sec<=6'b000000; // 秒 59 → 00, 启用分钟+1
            carry_sec<=1'b0; // 禁用分钟计数器+1
        end
    end
```

秒计数器模块续：

```
else if (count_sec==6'd58 && load_sec==0) begin
    count_sec<=count_sec+1;    // 58→59
    carry_sec1<=1'b0;          // 禁用分钟进位控制信号, 00:59:59 →
01:00:00
    carry_sec<=1'b1;          // 启用分钟计数器+1
end

else if (count_sec==6'd57 && load_sec==0) begin
    count_sec<=count_sec+1;    // 57 → 58
    carry_sec1<=1'b1;          // 启用分钟进位控制信号, 00:59:59
→01:00:00
end

else if (count_sec<6'd58 && enable_sec && load_sec==0) begin    // 秒计数器 + 1
    count_sec<=count_sec+1;
    carry_sec<=1'b0;          // 禁用分钟计数器+1
    end
    end
endmodule
```

## 2、分计数器模块

```
module counter_min(
    input clock,
    input reset_min,
    input enable_min,
    input enable_min1,
    input load_min,
    input setting_min,
    input [5:0] data_min,
    output reg [5:0]count_min,
    output reg carry_min
);
always @ (posedge clock or posedge reset_min)
begin
    if (reset_min)begin    // 清零
        count_min<=6'b000000;
        carry_min<=1'b0;
    end
    else if (load_min && setting_min && count_min<6'd59) begin
        count_min<=count_min+1;    // load+1=min+1
    end
end
```

分计数器模块续:

```
        else if (load_min && setting_min && count_min==6'd59) begin
            count_min<=6'b000000;    // load+1=59→00
        end

        else if (count_min==6'd59 && enable_min1 && load_min==0 ) begin    // min=59
            carry_min<=1'b1;        // counter_hour +1
        end

        else if (count_min==6'd59 && enable_min && load_min==0) begin
            count_min<=6'b000000;        // 00:59:59 → 01:00:00
            carry_min<=1'b0;            // 禁用 counter_min +1
        end

        else if (count_min==0 && enable_min==0 && load_min==0) begin
            carry_min<=1'b0;            // 禁用 counter_min +1
        end

        else if (count_min<6'd59 && enable_min && load_min==0) begin
            count_min<=count_min+1;    // counter_min +1
            carry_min<=1'b0;            // 禁用 counter_min +1
        end
    end
end
endmodule
```

### 3、小时计数器模块

```
module counter_hour(
    input clock,
    input reset_hour,
    input enable_hour,
    input load_hour,
    input setting_hour,
    input [5:0] data_hour,
    output reg [5:0] count_hour,
    output reg carry_hour
);
always @ (posedge clock or posedge reset_hour)
begin
    if (reset_hour) begin    // 清零
        count_hour<=6'b000000;
        carry_hour<=1'b0;
    end
end
```

#### 小时计数器模块续：

```
        else if (load_hour && setting_hour && count_hour < 6'd23) begin
            count_hour <= count_hour + 1;           // load+1=sec+1
        end

        else if (load_hour && setting_hour && count_hour == 6'd23) begin
            count_hour <= 6'b0000000;           // load hour +1 = 23 → 00
        end

        else if (count_hour == 6'd23 && enable_hour && load_hour == 0) begin
            count_hour <= 6'b0000000;           // hour 23 → 0
            carry_hour <= 1'b1;                 // day+1
        end

        else if (count_hour > 6'd23 && load_hour == 0) begin
            count_hour <= 6'b0000000;
            carry_hour <= 1'b0;                 // 禁用 carry+hour
        end

        else if (count_hour < 6'd23 && enable_hour && load_hour == 0) begin
            count_hour <= count_hour + 1;
            carry_hour <= 1'b0;                 // 禁用 carry+hour
        end
    end
end
endmodule
```

#### 4、顶层模块

```
module top_one(
    input clock,    // 时钟信号
    input reset_n,  // 复位信号，低电平有效
    input enable,   // 使能信号，高电平有效
    input load,     // 加载时间数据模式信号，高电平有效
    input setting1, // 加载秒数据信号，高电平有效
    input setting2, // 加载分钟数据信号，高电平有效
    input setting3, // 加载小时数据信号，高电平有效
    input [5:0] data_sec,    // 秒，范围为6'd0 ~ 6'd59
    input [5:0] data_min,    // 分，范围为6'd0 ~ 6'd59
    input [5:0] data_hour,   // 小时，范围为6'd0 ~ 6'd23
    output [5:0] count_sec,  // 秒输出
    output [5:0] count_min,  // 分输出
    output [5:0] count_hour, // 小时输出
    output carry_hour;       // 小时进位输出，用于表示天数+1
);
```

顶层模块续：

```
wire s_to_m;    // 连接: 秒进位信号到分钟输入使能信号 (模块 sec 到模块 min)
wire s1_to_m;   // 连接: 秒进位 1 信号到分钟输入 1 使能信号 (模块 sec 到模块 min)

wire m_to_h;    // 连接: 分钟进位信号到小时输入使能信号 (模块 min 到模块 hour)

counter_sec go1(
    .clock(clock),
    .reset_sec(reset_n),
    .enable_sec(enable),
    .load_sec(load),
    .setting_sec(setting1),
    .data_sec(data_sec),
    .count_sec(count_sec),
    .carry_sec(s_to_m),
    .carry_sec1(s1_to_m)
);

counter_min go2(
    .clock(clock),
    .reset_min(reset_n),
    .enable_min(s_to_m),
    .enable_min1(s1_to_m),
    .load_min(load),
    .setting_min(setting2),
    .data_min(data_min),
    .count_min(count_min),
    .carry_min(m_to_h)
);

counter_hour go3(
    .clock(clock),
    .reset_hour(reset_n),
    .enable_hour(m_to_h),
    .load_hour(load),
    .setting_hour(setting3),
    .data_hour(data_hour),
    .count_hour(count_hour),
    .carry_hour(carry_hour)
);
endmodule
```

## 5、仿真模块 testb

```
module testb();
    reg clock;
    reg reset_n;
    reg enable;
    reg load;
    reg [5:0] data_sec;
    reg [5:0] data_min;
    reg [5:0] data_hour;
    wire [5:0] count_sec;
    wire [5:0] count_min;
    wire [5:0] count_hour;
    wire carry_hour;
    top_one myclock (
        .clock(clock),
        .reset_n(reset_n),
        .enable(enable),
        .load(load),
        .data_sec(data_sec),
        .count_sec(count_sec),
        .data_min(data_min),
        .count_min(count_min),
        .data_hour(data_hour),
        .count_hour(count_hour),
        .carry_hour(carry_hour)
    );
    initial begin
        clock = 0;
        reset_n = 1;
        enable = 1;
        load = 0;
        data_sec = 0;
        data_min = 0;
        data_hour = 0;
        #10;
        reset_n = 0;
    end
    always begin
        clock = ~clock;
        #0.001;
    end
    initial
        #1000 $finish;
endmodule
```

## 6、分频模块

```
module clk_divider (  
    input clock_in,      // 输入时钟 100MHz  
    input reset,         // 低电平复位  
    output reg clock_out // 输出 1Hz 时钟  
);  
    reg [24:0] counter; // 25 位计数器  
    always @(posedge clock_in or negedge reset) begin  
        if (!reset) begin  
            counter <= 0;  
            clock_out <= 0; // 初始化输出时钟  
        end else if (counter == 25'd24999999) begin // 分频到 1Hz  
            counter <= 0;  
            clock_out <= ~clock_out; // 翻转输出时钟  
        end else begin  
            counter <= counter + 1;  
        end  
    end  
endmodule
```

7、约束文件 mycons：见附录；

为了便于实验，修改顶层代码，见附录；

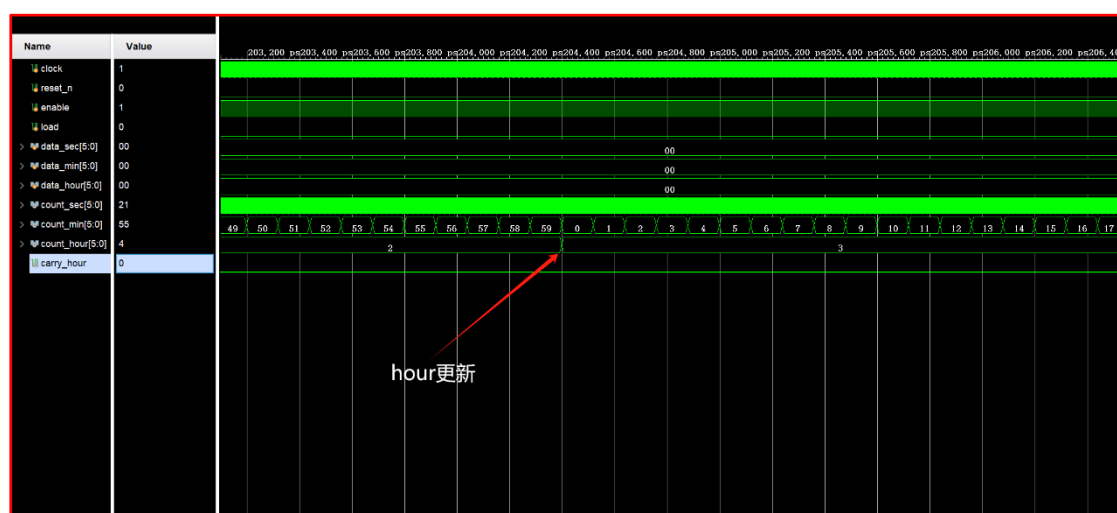
## 三、波形效果

1、如图所示，count\_hour 由 23→24 时，会重置为 0，且 carry\_hour（hour 的进位，相当于 day）产生一个脉冲。

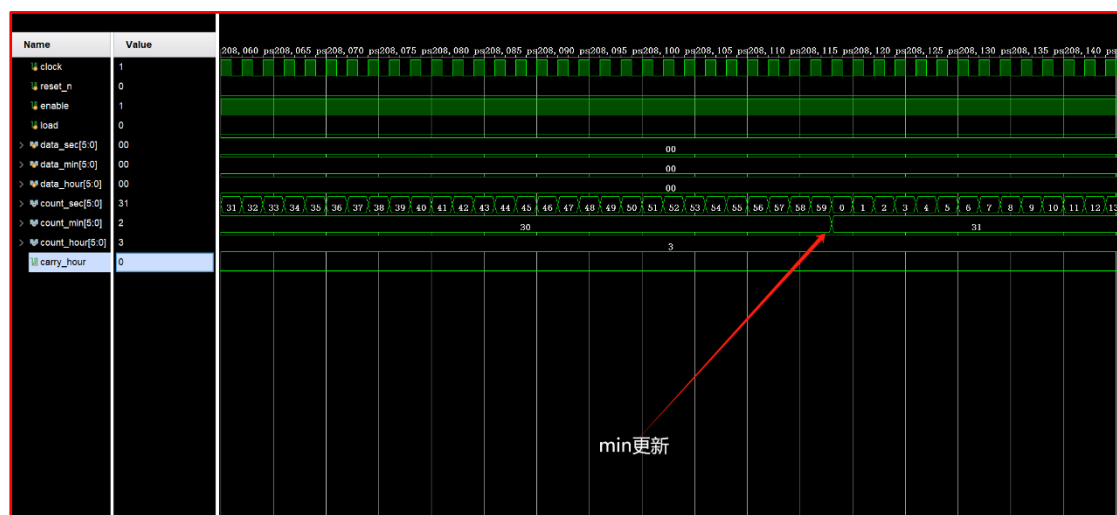


2、如图所示，count\_min 由 59→60 时，会重置为 0，且 count\_hour + 1。

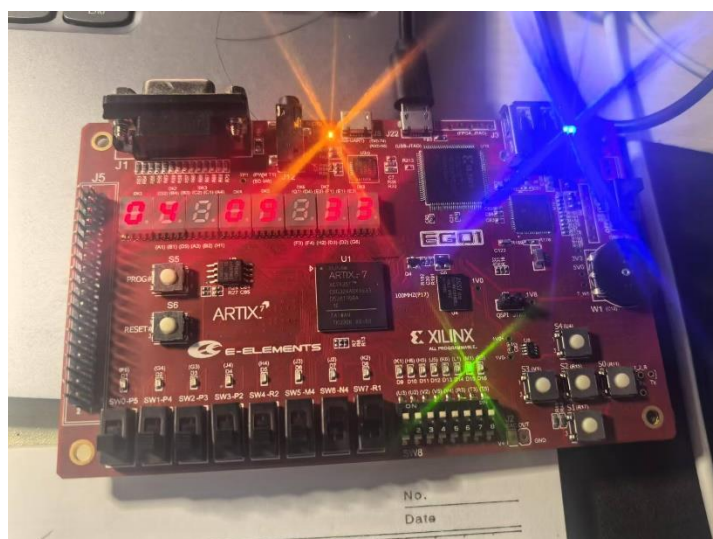




3、如图所示，count\_sec 由 59→60 时，会重置为 0，且 count\_min + 1。  
由上面的波形图可知成功实现了时分秒数字时钟。



#### 四、口袋实验板结果



如图所示，分别显示时、分、秒；五个开关分别控制重置、使能、三个时间单位的初始设置；亮起的 LED 灯指示使能开关置于高电平，另外有一个 LED 灯指示天数+1。

## 附录

### 修改后的顶层模块

```
`timescale 1ns / 1ps

module top_one(
    input clock,                // 时钟信号, 100MHz
    input reset_n,              // 复位信号, 低电平有效
    input enable,
    input load_sec,
    input load_min,
    input load_hour,
    output reg [7:0] seg,        // 段选信号 (数码管)
    output reg [7:0] seg1,       // 用于秒计时的段选信号
    output reg [3:0] bit_select, // 位选信号 (动态扫描)
    output reg [3:0] bit_select1, // 位选信号 (动态扫描)
    output debug_clk_1hz,
    output carry_day
);

    // 分频器模块, 生成 1Hz 的时钟信号
    wire clock_1hz;

    clk_divider clk_gen (
        .clock_in(clock),
        .reset(reset_n),
        .clock_out(clock_1hz)
    );
    assign debug_clk_1hz = enable;
    // 中间信号
    wire s_to_m;    // 秒进位到分钟
    wire s1_to_m;   // 秒进位信号到分钟
    wire m_to_h;    // 分钟进位到小时
    wire reset;
    assign reset = ~reset_n; // 转换低电平有效的复位为高电平有效

    // 秒、分、小时数据寄存器
    reg [5:0] data_sec;
    reg [5:0] data_min;
    reg [5:0] data_hour;
    wire [5:0] count_sec;
    wire [5:0] count_min;
```

```

wire [5:0] count_hour;

// 复位或计数更新
always @(posedge clock_1hz or negedge reset_n) begin
    if (!reset_n) begin
        data_sec <= 6'b0;
        data_min <= 6'b0;
        data_hour <= 6'b0;

    end
end

```

```

// 秒计数模块
counter_sec go1(
    .clock(clock_1hz),
    .reset_sec(reset),
    .enable_sec(enable),
    .load_sec(load_sec),
    .setting_sec(load_sec),
    .data_sec(data_sec),
    .count_sec(count_sec),
    .carry_sec(s_to_m),
    .carry_sec1(s1_to_m)
);

```

```

// 分钟计数模块
counter_min go2(
    .clock(clock_1hz),
    .reset_min(reset),
    .enable_min(s_to_m),
    .enable_min1(s1_to_m),
    .load_min(load_min),
    .setting_min(load_min),
    .data_min(data_min),
    .count_min(count_min),
    .carry_min(m_to_h)
);

```

```

// 小时计数模块
counter_hour go3(
    .clock(clock_1hz),
    .reset_hour(reset),
    .enable_hour(m_to_h),

```

```

        .load_hour(load_hour),
        .setting_hour(load_hour),
        .data_hour(data_hour),
        .count_hour(count_hour),
        .carry_hour(carry_day)
    );

    // 动态扫描逻辑
    reg [1:0] scan_pos = 0;
    reg [19:0] scan_clk_div = 0;
    reg scan_clk = 0;

    // 扫描时钟生成: 100MHz -> 1kHz
    always @(posedge clock or negedge reset_n) begin
        if (!reset_n) begin
            scan_clk_div <= 0;
            scan_clk <= 0;
        end else if (scan_clk_div == 18'd49_999) begin // 修改分频值
            scan_clk_div <= 0;
            scan_clk <= ~scan_clk;
        end else begin
            scan_clk_div <= scan_clk_div + 1;
        end
    end

    // 扫描位置更新
    always @(posedge scan_clk or negedge reset_n) begin
        if (!reset_n) // 低电平复位
            scan_pos <= 0;
        else
            scan_pos <= scan_pos + 1;
    end

    // 当前扫描位的数字选择
    reg [3:0] current_digit;
    reg [3:0] current_digit1;
    always @(*) begin
        current_digit = 4'd0;
        current_digit1 = 4'd0;
        case (scan_pos)
            2'd0: begin
                current_digit1 = count_min % 10; // 分个位
                current_digit = count_hour / 10; // 时十位
            end
        end
    end

```

```

        end
        2'd1: begin
            current_digit1 = 4'b1111;
            current_digit = count_hour % 10;
        end
        2'd2: begin
            current_digit = 4'b1111;
            current_digit1 = count_sec / 10;
        end
        2'd3: begin
            current_digit1 = count_min / 10;
            current_digit1 = count_sec % 10;
        end
    endcase
end

// 段选信号生成
always @(*) begin
    case (current_digit)
        4'd0: seg = 8'b0011_1111;
        4'd1: seg = 8'b0000_0110;
        4'd2: seg = 8'b0101_1011;
        4'd3: seg = 8'b0100_1111;
        4'd4: seg = 8'b0110_0110;
        4'd5: seg = 8'b0110_1101;
        4'd6: seg = 8'b0111_1101;
        4'd7: seg = 8'b0000_0111;
        4'd8: seg = 8'b0111_1111;
        4'd9: seg = 8'b0110_1111;
        default: seg = 8'b0000_0000; // 默认熄灭
    endcase
end

always @(*) begin
    case (current_digit1)
        4'd0: seg1 = 8'b0011_1111;
        4'd1: seg1 = 8'b0000_0110;
        4'd2: seg1 = 8'b0101_1011;
        4'd3: seg1 = 8'b0100_1111;
        4'd4: seg1 = 8'b0110_0110;
        4'd5: seg1 = 8'b0110_1101;
        4'd6: seg1 = 8'b0111_1101;
        4'd7: seg1 = 8'b0000_0111;
        4'd8: seg1 = 8'b0111_1111;

```

```

        4'd9: seg1 = 8'b0110_1111;
        default: seg1 = 8'b0000_0000; // 默认熄灭
    endcase
end

// 位选信号生成
always @(*) begin
    bit_select = 4'b0000;
    bit_select1 = 4'b0000;
    case (scan_pos)
        2'd0: begin
            bit_select1 = 4'b0001; // 秒个位
            bit_select = 4'b0001; // 分十位
        end
        2'd1: begin
            bit_select1 = 4'b0010; // 秒十位
            bit_select = 4'b0010;
        end
        2'd2: begin
            bit_select = 4'b0100; // 时个位
            bit_select1 = 4'b0100; // 时个位
        end
        2'd3: begin
            bit_select1 = 4'b1000; // 分个位
            bit_select = 4'b1000; // 时十位
        end
    endcase
end
endmodule

```

#### 约束文件

##### # 时钟信号

```

set_property PACKAGE_PIN P17 [get_ports clock]
set_property IOSTANDARD LVCMOS33 [get_ports clock]

```

##### # 复位信号

```

set_property PACKAGE_PIN R1 [get_ports reset_n]
set_property IOSTANDARD LVCMOS33 [get_ports reset_n]

```

##### # 使能信号

```

set_property PACKAGE_PIN N4 [get_ports enable]
set_property IOSTANDARD LVCMOS33 [get_ports enable]

```

```

set_property PACKAGE_PIN M4 [get_ports load_sec]
set_property IOSTANDARD LVCMOS33 [get_ports load_sec]

```

```
set_property PACKAGE_PIN R2 [get_ports load_min]
set_property IOSTANDARD LVCMOS33 [get_ports load_min]
```

```
set_property PACKAGE_PIN P2 [get_ports load_hour]
set_property IOSTANDARD LVCMOS33 [get_ports load_hour]
```

```
set_property PACKAGE_PIN K3 [get_ports carry_day]
set_property IOSTANDARD LVCMOS33 [get_ports carry_day]
```

```
set_property PACKAGE_PIN M1 [get_ports debug_clk_1hz]
set_property IOSTANDARD LVCMOS33 [get_ports debug_clk_1hz]
```

#### # 数码管段选信号绑定

```
set_property PACKAGE_PIN B4 [get_ports {seg[0]}]
set_property PACKAGE_PIN A4 [get_ports {seg[1]}]
set_property PACKAGE_PIN A3 [get_ports {seg[2]}]
set_property PACKAGE_PIN B1 [get_ports {seg[3]}]
set_property PACKAGE_PIN A1 [get_ports {seg[4]}]
set_property PACKAGE_PIN B3 [get_ports {seg[5]}]
set_property PACKAGE_PIN B2 [get_ports {seg[6]}]
set_property PACKAGE_PIN D5 [get_ports {seg[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[*]}]
```

#### # 数码管位选信号绑定

```
set_property PACKAGE_PIN G2 [get_ports {bit_select[0]}]
set_property PACKAGE_PIN C2 [get_ports {bit_select[1]}]
set_property PACKAGE_PIN C1 [get_ports {bit_select[2]}]
set_property PACKAGE_PIN H1 [get_ports {bit_select[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {bit_select[*]}]
```

#### # 数码管段选信号绑定（用于秒计时）

```
set_property PACKAGE_PIN D4 [get_ports {seg1[0]}]
set_property PACKAGE_PIN E3 [get_ports {seg1[1]}]
set_property PACKAGE_PIN D3 [get_ports {seg1[2]}]
set_property PACKAGE_PIN F4 [get_ports {seg1[3]}]
set_property PACKAGE_PIN F3 [get_ports {seg1[4]}]
set_property PACKAGE_PIN E2 [get_ports {seg1[5]}]
set_property PACKAGE_PIN D2 [get_ports {seg1[6]}]
set_property PACKAGE_PIN H2 [get_ports {seg1[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg1[*]}]
```

#### # 数码管位选信号绑定（用于秒计时）

```
set_property PACKAGE_PIN G1 [get_ports {bit_select1[0]}]
```

```
set_property PACKAGE_PIN F1 [get_ports {bit_select1[1]}]  
set_property PACKAGE_PIN E1 [get_ports {bit_select1[2]}]  
set_property PACKAGE_PIN G6 [get_ports {bit_select1[3]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {bit_select1[*]}]
```